

React Native Template

+ Claude

Onboarding & Developer Guide

Everything your team needs to build with confidence.

Expo SDK 53

TypeScript

React Query

Reanimated

Maestro E2E

Storybook

1. What Is This Template?

An Expo-managed React Native template pre-configured with a curated set of tools, conventions, and Claude AI integration. It gives your team a shared foundation so every developer — regardless of experience level — follows the same patterns from day one.

Core Stack	Testing & DX
Expo SDK 53 — managed workflow, OTA updates	Jest + RNTL — unit & component tests
TypeScript — strict mode enforced	Maestro — end-to-end device flows
React Query — server state & caching	ESLint + Prettier — consistent formatting
Reanimated — 60fps animations	Path aliases — @/ imports throughout
Storybook — component documentation	Claude AI — enforced conventions

2. Project Structure

The project follows a clear, opinionated folder structure:

```
your-app/  
  
src/  
  
components/ # Shared UI components (used by 2+ screens)  
  
constants/ # App-wide constants  
  
hooks/ # Custom React hooks  
  
screens/ # Screen-level components + local components  
  
theme/ # Design tokens: colors, typography, spacing  
  
utils/ # Pure utility functions  
  
e2e/ # Maestro end-to-end test flows  
  
scripts/ # Shell scripts for setup and CI  
  
.claude/ # Claude AI configuration  
  
rules/ # Code conventions Claude enforces  
  
commands/ # Custom slash commands (/deploy, /setup)
```

Key principle: Every folder has a single responsibility. Claude knows this structure and will always place new files in the correct location.

3. What Was Added & Modified

The following changes were made to enable Claude integration and enforce team conventions:

File / Folder	What Changed	Why
tsconfig.json	Added baseUrl + @/* paths	Enables @/ imports across all src/ files
CLAUDE.md	Created — team instructions	Claude reads this on every session start
.claude/rules/ code-style.md	Created	No barrels, @/ aliases, no inline styles, named exports
.claude/rules/ testing.md	Created	Jest colocation, naming, jestWrapper, Maestro rules
.claude/rules/ rn-conventions.md	Created	Component tiers, hooks, theme, accessibility
.claude/commands/ deploy.md	Created	Defines the /deploy slash command
.claude/commands/ setup.md	Created	Defines the /setup interactive library wizard
scripts/ qa-branch-switch.sh	Created	Full branch switch workflow for QA engineers

4. Path Aliases — @/ Imports

tsconfig.json now maps @/* to src/*. You never need to count dots in relative imports. Expo + Metro resolve these natively — no Babel plugin needed.

Good	Bad
import {useTheme} from '@/hooks/useTheme';	import {useTheme} from '../../../../hooks/useTheme';
import {testIds} from '@/utils/testIds';	import {testIds} from '../../../../utils/testIds';
import {HomeScreen} from '@/screens/HomeScreen';	import {HomeScreen} from '../screens/HomeScreen';

5. Component Architecture

Components follow a two-tier model based on how broadly they are used:

Tier	Location	When to use	Example
Shared	src/components/	Used in 2+ screens	Button, Card, Avatar
Local	src/screens//components/	Used in 1 screen only	HeroBanner, StatsRow

Shared component structure (with tests/stories):

```
src/components/Button/

Button.tsx # component

Button.test.tsx # unit tests

Button.stories.tsx # Storybook stories (optional)
```

Local component structure (screen-scoped):

```
src/screens/

HomeScreen.tsx

HomeScreen.test.tsx

components/ # only used by HomeScreen

HeroBanner.tsx

StatsRow.tsx
```

Prefix convention: When Claude creates a shared component it will ask if you want a prefix (e.g. App, UI, Base). If you choose one it will be applied consistently: AppButton, AppCard, AppAvatar. You can skip the prefix and use plain names.

6. Code Style Rules

No Barrel Files

Do NOT create index.ts files that re-export everything from a folder. Import directly from the source file. Exception: src/theme/index.ts is intentional — it is the public API of a cohesive module.

No Inline Styles

Never write style={{...}} directly on a component. Always use StyleSheet.create() or theme tokens. Dynamic values like colors can be mixed in as a second array item: [styles.container, {backgroundColor: theme.bg}].

No any Type

strict: true is enforced. Use unknown + type narrowing instead of any. Use interface for object shapes, type for unions and utility types. Prefer explicit return types on exported functions.

Named Exports Only

Use named exports everywhere. Default exports are only allowed for the root App component.

7. Testing Conventions

Two independent testing layers — each used for a different purpose:

Layer	Tool	Location	Tests what?
Unit / Component	Jest + RNTL	Next to source file	Logic, rendering, interactions
End-to-End	Maestro	e2e/ folder	Full user flows on device

Unit test naming convention:

```
describe('TodoCount', () => {  
  
  it('should render the count returned by the API', () => { ... });  
  
  it('should show a loading state while fetching', () => { ... });  
  
});
```

Tests that use React Query must use jestWrapper:

```
import {jestWrapper} from '@utils/jestWrapper';  
  
render(, {wrapper: jestWrapper});
```

Maestro e2e flow structure:

```
appId: com.yourorg.yourapp  
  
---  
  
- launchApp:  
  
  arguments:  
  
  isE2E: true # disables LogBox  
  
- assertVisible: 'Hello World!'  
  
- tapOn: 'Sign In'
```

8. Claude AI Integration

Claude Code is configured to understand this project's conventions automatically. Every session, Claude reads CLAUDE.md and all files in .claude/rules/ before writing any code. You don't need to repeat instructions — Claude already knows the rules.

```
.claude/

rules/ # Claude reads these on every session

code-style.md # TypeScript, imports, naming, styling

testing.md # Jest + Maestro conventions

rn-conventions.md # Component structure, hooks, theme

commands/ # Slash commands you trigger manually

deploy.md # /deploy – runs prebuild for iOS/Android

setup.md # /setup – interactive library wizard
```

Available Slash Commands:

Command	What it does
<code>/deploy</code>	Asks for target platform (iOS / Android / both), runs scripts/prebuild.sh, verifies native output, and tells you what to run next.
<code>/setup</code>	Interactive wizard: reads your package.json, asks 6 questions about your project, suggests libraries by category with justification. Nothing is installed until you confirm.

Important: Claude will never install libraries or make breaking changes without your confirmation. The `/setup` wizard has an explicit confirmation step before running any installs.

9. How to Use Claude Day-to-Day

Claude Code runs as an AI assistant directly in your terminal. It reads the project conventions automatically on every session — you do not need to explain the rules each time.

Starting a session:

```
# Open Claude Code in the project root

cd your-project

claude

# Claude will automatically read:

# CLAUDE.md – team instructions
```

```
# .claude/rules/ – code conventions

# .claude/commands/ – available slash commands
```

Triggering slash commands:

Type a slash command directly in the Claude chat. Claude will execute the workflow defined in `.claude/commands/`.

```
/deploy

# Claude asks: which platform? (ios / android / both)

# Then runs: ./scripts/prebuild.sh --platform ios

# Then confirms native output and prints next steps


/setup

# Claude reads package.json, asks 6 questions about your app,

# suggests libraries by category. Nothing installs until you confirm.
```

Practical prompt examples for this template:

What you want	What to type to Claude
Create a shared component	"Create a Card component that receives title and subtitle as props"
Create a screen-local component	"Add a HeroBanner component to the HomeScreen, local only"
Create a custom hook	"Create a useUserProfile hook that fetches from /api/profile"
Add a new screen	"Create a ProfileScreen with a loading state and error state"
Write unit tests	"Write tests for the TodoCount component"
Add a Maestro e2e flow	"Create a Maestro flow for the login screen"
Review code for violations	"Review HomeScreen.tsx for any convention violations"
Add a library	"Add Zustand for global UI state"

What Claude does automatically (without being asked):

Behavior	Example
Places files in the right folder	A shared Button goes to <code>src/components/Button/Button.tsx</code>
Uses <code>@/</code> path aliases	<code>import {useTheme} from '@/hooks/useTheme' — never relative</code>
Avoids barrel files	Will not create <code>index.ts</code> re-exports
Uses <code>StyleSheet.create()</code>	Will not write <code>style={{...}}</code> inline

Wraps React Query calls	Creates a named hook instead of calling useQuery in a component
Asks about component prefix	Prompts: "Do you want a prefix? e.g. App, UI, Base"
Collocates test files	Creates Component.test.tsx next to Component.tsx
Uses theme tokens	Calls useTheme() — never hardcodes '#ffff'

Tip: You can ask Claude to explain any decision it makes. For example: "Why did you put this file in src/components instead of the screen folder?" Claude will reference the specific rule from .claude/rules/ that guided the choice.

10. Scripts Reference

Script	Purpose	Audience
./scripts/init.sh	Full clean setup from scratch. Deletes node_modules, ios, android and reinstalls everything.	Devs — first setup
./scripts/install-dependencies.sh	Detects yarn/npm from lock file and installs dependencies.	Called by other scripts
./scripts/prebuild.sh	Runs expo prebuild for iOS / Android. Accepts --platform ios android.	Devs + CI
./scripts/qa-branch-switch.sh	Full branch switch for QA: validates env, backs up state, switches branch, cleans all caches, reinstalls deps, rebuilds native projects, verifies.	QA Engineers

qa-branch-switch.sh usage:

```
./scripts/qa-branch-switch.sh feature/login

./scripts/qa-branch-switch.sh --debug hotfix/payment

./scripts/qa-branch-switch.sh --dry-run develop

./scripts/qa-branch-switch.sh --clean-ports

./scripts/qa-branch-switch.sh --check-auth
```

Step	Action
1	Validate environment (node, yarn/npm, git, Expo CLI)
2	Check GitHub package registry authentication if needed
3	Backup current branch + uncommitted changes
4	Fetch latest from origin + switch to target branch
5	Deep clean: Metro, Watchman, Gradle, Expo caches, node_modules

6	Reinstall dependencies via install-dependencies.sh
7	Rebuild native projects via prebuild.sh
8	Install CocoaPods (iOS) with automatic retry
9	Post-switch validation: package.json, deps, native projects
10	Print run instructions: yarn ios / yarn android / yarn start

11. Daily Commands Cheatsheet

Command	Purpose
<code>yarn start</code>	Start Metro bundler
<code>yarn ios</code>	Run on iOS device
<code>yarn android</code>	Run on Android device
<code>yarn test</code>	Run Jest unit tests
<code>yarn test:e2e</code>	Run Maestro e2e tests
<code>yarn lint</code>	ESLint check
<code>yarn format</code>	Prettier format
<code>yarn start:storybook</code>	Launch Storybook
<code>/deploy</code>	Claude: run prebuild for a platform
<code>/setup</code>	Claude: interactive library suggestion wizard

12. QA Engineer Onboarding

QA engineers should never manually switch branches. Always use the branch switch script to avoid stale cache issues and broken native builds.

First time setup:

```
# 1. Clone the repository

git clone && cd

# 2. Run full setup

./scripts/init.sh
```

```
# 3. Start the app

yarn ios # or: yarn android
```

Switching branches (daily workflow):

```
# Switch to a feature branch

./scripts/qa-branch-switch.sh feature/new-login

# With debug output

./scripts/qa-branch-switch.sh --debug hotfix/payment

# Dry run – see what would happen

./scripts/qa-branch-switch.sh --dry-run develop

# If Metro ports are stuck

./scripts/qa-branch-switch.sh --clean-ports
```

If something fails: Check the log file at the project root: **qa-branch-switch.log**. The script will also attempt to restore your original branch automatically on error.

13. Quick Reference Card

Topic	Rule
Component — shared	Used in 2+ screens → src/components/ (Claude asks about prefix)
Component — local	Used in 1 screen → src/screens//components/
Imports	Always @/ — never relative paths going up more than one level
Barrel files	Never create index.ts re-exports (except src/theme/index.ts)
Styles	Always StyleSheet.create() — never style={{...}} inline
Colors	Always theme tokens via useTheme() — never hardcoded hex values
TypeScript	No any — use unknown + narrowing. Named exports only
Hooks	Always use prefix, one per file, live in src/hooks/
Data fetching	Never call useQuery directly in components — wrap in a named hook
Animations	Always Reanimated — never the built-in Animated API

Tests — unit	Colocated next to source file, named Component.test.tsx
Tests — e2e	Maestro flows in e2e/, assert on visible text not testIDs

Generated for the React Native Template team. Keep this document updated when conventions change.